

Modul 07: Forms

<i>Ziel:</i>	<i>Formulare erstellen</i>
<i>Tools:</i>	<i>Visual Studio Code, Node.js</i>
<i>Dauer:</i>	<i>~45min</i>
<i>Autor:</i>	<i>Ortwin Carstoea</i>
<i>Letzte Änderung:</i>	<i>05.08.2024</i>

1 Reaktives Formular erstellen

Wir wandeln die task-form Komponente in ein reaktives Formular um. Hierfür müssen wir zuerst das `ReactiveFormsModule` importieren. Wenn wir Module verwenden, muss es den Imports in `app.module.ts` hinzugefügt werden.

```
import { ReactiveFormsModule } from '@angular/forms';

// ...
imports: [BrowserModule, AppRoutingModule, ReactiveFormsModule],
// ...
```

Im nächsten Schritt holen wir uns den `FormBuilder` als Abhängigkeit in unsere Klasse hinein. Wir deklarieren ein Feld `formGroup` vom Typ `FormGroup` und initialisieren es danach.

```
ngOnInit(): void {
  const task = this.taskService.getTask(this.id ?? '') ?? <TaskItem>{};
  const dueDate = task.dueDate
    ? DateTime.fromJSDate(task.dueDate).toISODate() : '';
  const labels = task.labels
    ? task.labels.map((e) => this.formBuilder.control(e)) : [];
  this.formGroup = this.formBuilder.group({
    id: { value: task.id, disabled: true },
    title: [task.title, Validators.required],
    dueDate: dueDate,
    priority: task.priority,
    labels: this.formBuilder.array(labels),
  });
}
```

Mit diesem Builder erstellen wir uns ein Formular mit allen nötigen Gültigkeitsregeln. Da die `formGroup` im Initialzustand undefiniert ist, müssen wir in der HTML-Vorlage es vor der Verwendung prüfen. Wir benutzen die Direktiven `formGroup` und `formControlName` um die HTML-Elementen und dem erstellten Form-Modell zu binden.

```

@if (formGroup) {
  <form class="grid" [formGroup]="formGroup" (ngSubmit)="submit()">
    <label for="Id">Id</label>
    <input id="id" type="text" formControlName="id" />
    <label for="title">Aufgabe</label>
    <input id="title" type="text" required formControlName="title" />
    <label for="dueDate">Fälligkeit</label>
    <input id="dueDate" type="date" formControlName="dueDate" />
    <label for="priority">Priorität</label>
    <div id="priority">
      <span title="Niedrig"><input id="0" type="radio"
        formControlName="priority" value="low" /> ↓</span>
      <span title="Normal"><input id="1" type="radio"
        formControlName="priority" value="default" /> ~</span>
      <span title="Wichtig"><input id="2" type="radio"
        formControlName="priority" value="important" /> !</span>
    </div>
    <label for="labels">
      <button (click)="addLabel()">Labels +</button>
    </label>
    <div class="labels" formArrayName="labels">
      @for (label of labels.controls; track $index) {
        <input type="text" [formControlName]="$index" />
      }
    </div>
    <button type="submit" [disabled]="!formGroup.valid">
      @if (editing) { Speichern } @else { Neue Aufgabe }
    </button>
  </form>
} @else {
  <p>Lade gerade '{{ id }}'</p>
}

```

Die Template Reference Variables sind völlig entfallen. Die Labels erhalten wir von einem getter und fügen neue mit der Methode `addLabel()` hinzu.

```

get labels(): FormArray {
  return <FormArray>this.formGroup?.get('labels');
}

addLabel() {
  this.labels.push(this.formBuilder.control(''));
}

```

Zudem ist zu beachten, dass wir nun nur noch einen Button vom Typ `submit` haben sowie die Direktive `ngSubmit` verwenden, um die Methode `submit()` aufzurufen. Die angepasste Methode sind nun so aus:

```

submit() {
  const task: TaskItem = {
    id: this.id || '',
    title: this.formGroup!.value.title!,
    dueDate: this.formGroup!.value.dueDate
      ? DateTime.fromISO(this.formGroup!.value.dueDate).toJSDate()
      : undefined,
    priority: 'default',
    labels: this.labels.value,
    completed: false,
  };

  if (this.editing) {
    this.taskService.updateTask(this.id!, task);
    this.toastService.sendInfo(`${task.title} wurde gespeichert`);
  } else {
    this.taskService.addTask(task);
    const list = this.taskService.items();
    this.toastService.sendInfo(`${list.length} Aufgaben insgesamt`);
  }

  this.router.navigate(['/list']);
}

```

Wir holen uns die Werte aus dem FormGroup-Modell und erstellen daraus unsere Aufgabe, welche wir dann an den Service schicken.

2 Template getriebenes Formular

Wir wollen auch das Template getriebene Formular ausprobieren. Hierzu erstellen wir eine neue Komponente `components/task-quick`, welches nur das Textfeld für den Titel enthalten soll. Diese Komponente integrieren wir in unsere `routes/task-list`, so wie es ursprünglich mal der Fall war.

Dieses Mal importieren wir in `app.module.ts` das `FormsModule` aus demselben Package wie das `ReactiveFormsModule`.

Im Code-Behind soll es ein Feld für das TaskItem und eine Methode `submit()` geben, welche die Aufgabe anlegt. Mit der `[(ngModel)]` Direktive können wir Objekte und Primitives sehr einfach binden. Die Runden und eckigen Klammern weisen auf die Two-Way-Datenbindung hin.

```

<div class="grid">
  <input type="text" placeholder="Neue Aufgabe"
    [(ngModel)]="task.title" (keyup.enter)="submit()" />
  <button (click)="submit()">✓</button>
</div>

```

3 Inline-Editing für bestehende Aufgaben

Wenn der Benutzer auf den Aufgabentext klickt, soll ich der Text in ein Input-Element umwandeln. Es soll dafür die soeben angelegte task-quick Komponente verwendet und erweitert werden.

`<ng-content />` bietet sich sehr gut an um den Inhalt im Darstellungsmodus innerhalb der Komponente anzuzeigen und gleichzeitig die Formatierung beizubehalten.

- Aufgabe hinzufügen

☐
! Angular Kurs absolvieren (in 1 Tag)






- Aufgabe bearbeiten

☐
! Angular Kurs absolvieren (in 1 Tag)




☐






